

CONFIDENTIAL



Command Line Arbitrary Waveform Encoder (CLAW) Version 2.1 User Manual

John A. Flynn
revised: 8/14/2019
Verasonics, Inc

Command Line Arbitrary Waveform Encoder (CLAWE) v2.1

Contents

1	Introduction	page 2
2	References / Companion Documents	page 3
3	Quick Start Guide	page 4
4	System Requirements	page 4
5	Toolbox Components New to Encoder Version 2	page 4
6	Theory: Processing Kernel and Gain Search	page 6
7	Transmitter Capacity Considerations	page 6
8	Example Encodings	page 7
	Appedix: Detailed Function Descriptions	page 10

1. Introduction

This document explains usage and components of the Command Line Arbitrary Waveform Encoder (**CLAWE**) version 2.1. This tool allows the user to create tri-state transmitter sequences designed to approximate user-specified analog acoustic or received waveforms. The CLAWE upgrades functionality over the previous Arbitrary Waveform Generation Toolbox GUI tool (**GUI-AWE**) version v1, which is described in the document titled “Arbitrary Waveform Generation Toolbox”. The improved functionality consists of **significantly better encoding performance**. For usage of the existing GUI-AWE please refer to its documentation.

Please see Section 2 (**References/Companion Documents**) for the filenames of relevant documents.

The underlying operating principle of the encoder (as in the previous version) is to exploit the filtering inherent in transducer dynamics in order to temporally smooth the output of the tri-state transmitter, in a way that approximates a desired continuous-valued signal. The technology of CLAWE is based on optimal sequence estimation efficiently computed by the Viterbi algorithm.

The performance of the CLAWE is significantly improved, the existing GUI-based Arbitrary Waveform Encoder (and its command line examples) remain a functional tool for rapid prototyping of encodings, as its computation time can be much faster than the CLAWE.

2. Helpful References / Companion Documents

For background on the Arbitrary Waveform (**AW**) encoding problem in context of the Verasonics system, including usage models, please refer to the following documents:

New Command-Line Arbitrary Waveform Encoder (CLAW) version v2 (this document):

file: ArbWaveEncoder_v2_user_manual.pdf

Flynn, John A. "Command Line Arbitrary Waveform Encoder (CLAW) Version 2.1 User Manual"

Arbitrary Waveform Generation Toolbox GUI tool (GUI-AWE) version v1:

file: ArbWaveToolbox_v1_Documentation.pdf

Verasonics staff, "Arbitrary Waveform Generation Toolbox"

USPTO Patent application for AW encoding methods including Viterbi:

file: ArbWave-patent-app-WO2015009960.pdf

Flynn, John A., et al. "Method and system for arbitrary waveform generation using a tri-state transmit pulser." U.S. Patent Application Nos. 14/905,434.

UFFC-IUS presentation explaining linear AW encoding and usage models:

file: IEEE-2013_Verasonics-arbwave-presentation.pdf

Flynn, John A., et al. "Arbitrary waveforms using a tri-state transmit pulser." *Ultrasonics Symposium (IUS), 2013 IEEE International, Session IUS1-A3-4 Medical Signal Processing* (presentation materials).

UFFC-IUS conference paper on linear AW encoding methods:

<http://ieeexplore.ieee.org/document/6725161/>

Flynn, John A., et al. "Arbitrary waveforms using a tri-state transmit pulser." *Ultrasonics Symposium (IUS), 2013 IEEE International* (pp 41-44). IEEE, 2013.

UFFC-IUS conference paper on Viterbi (nonlinear) AW encoding methods:

<https://ieeexplore.ieee.org/abstract/document/8579705>

Flynn, John A. "Improved Arbitrary Waveform Synthesis for Tri-State Transmitters by an Impulse Response Factorization Enabling Use of the Viterbi Algorithm." 2018 IEEE International Ultrasonics Symposium (IUS). IEEE, 2018..

Application Note on Use Models for Verasonics AW encoding methods

file: ArbWave_UseModels.pdf

Flynn, John A., "Use Models and Fidelity Criteria for Verasonics Arbitrary Waveform Encoders"

3. Quick Start Guide

Processing in the encoding example **encode_dp2.m** is summarized by the workflow:

- A. Define the “analog” reference waveform as a MatLab vector array of samples.
- B. Use **viterbi_factor2.p** to encode the reference waveform into a tri-state vector **x3**.
- C. Format **x3** as a TW ‘pulseCode’ type: `>> pc=twgen( 'trinary2gen3',x3)`
- D. Assign the pulse code array **pc** to the TW structure member **PulseCode**.

4. System Requirements

The CLAW version 2.1 requires that the Host Controller, or workstation running this toolbox, have installed the MathWorks (tm) Communications System Toolbox and its requisite components.

5. Toolbox Components New to AWE Toolbox Version 2.1

The following components in the AWE Toolbox are specific to the CLAW. They are described in the Appendix.

Documentation:

ArbWaveEncoder_v2_user_manual.pdf (This document) Command Line Arbitrary Waveform Encoder (CLAW) Version 2.1 User Manual.

WO2015009960.pdf Patent, International Application Number PCT/US2014/047080.

IEEE-2013_Verasonics-arbwave-presentation.pdf Presentation materials from UFFC/IEEE International Ultrasound Symposium, 2013.

MatLab functions and scripts:

arbwavdev2.m (reference waveform utility)
encode_dp2.m (top-level example for Viterbi encoding)
getIRinfo.m (retrieve empirical impulse response data for transducer)
nrmseVec_VS.p (error measurement between two vectors)
viterbi_symbol2.m (encoder algorithm parameter sets)
viterbi_factor2.p (pre-processing, signal conditioning, and invocation of encoder)

Transducer impulse response files:

cirest0701_code_acoustic-L74-23.mat (legacy format)
VerasonicsRoot/System/L11-5v_info.mat
VerasonicsRoot/System/L22-14vX_info.mat

Private Helper/Utility functions:

alignvec03, isodd, makeAlphabet

6. Theory: Processing Kernel and Gain Search

The purpose of the CLAW is to generate a tri-state pulse sequence in a manner that optimally approximates a specified continuous-valued reference waveform observed through the transducer, either hypothetically in the acoustic medium (one-way path usage model), or at reception (two-way path usage model). For a more detailed explanation of the **usage model** concept and background, please consult the bibliography materials (some of which are included in the documentation).

The Viterbi Algorithm (**VA**) is designed to accomplish this encoding as efficiently as possible. However, due to the typically long impulse responses of many transducers (in terms of transmit clock cycles), the approximation problem when naively stated remains intractable with VA. Therefore, an approximation is made by partitioning the impulse response into vectors of two lengths, a shorter vector appropriate for the VA and a long vector for linear processing, respectively.

The **gain search** exploits the assumption that any waveform achieved can be scaled to arbitrary levels through the power supply voltage setting. (The gain search vector is an input to the component viterbi_factor.p). Therefore, when transmit voltage is a free variable, the reference waveform scaling is a nuisance variable. The search process is:

Gain Search Process

1. scale the reference/design waveform by a candidate gain setting
2. compute a tri-state transmit encoding to approximate the scaled reference signal
3. assess an approximation error **Eapp** by comparing the scaled reference against the achieved transducer output produced by the candidate tri-state encoding.

The above steps are repeated over all candidate gains in the gain vector, and the best gain and tri-state encoding combination is chosen as that achieving the lowest **Eapp** error.

7. Transmitter Capacity Considerations

Depending on number of transmit channels and other Sequence Programming script properties, the arbitrary waveform encoder can generate tri-state sequences that exceed the high-voltage and gate driver power supplies' capabilities (as well as other limits). To evaluate the feasibility of a specific configuration created by a Sequence Programming script, it must be evaluated by running on a VDAS hardware system through the VSX command. This will automatically invoke **Limit Check** tests to qualify the sequence for hardware execution.

8. Examples

To help introduce the user to the new functionality of the CLAWE tools, the following examples are given in the file **encode_dp2.m** which is included in this toolbox.

Performance of the CLAWE Viterbi-Algorithm encoder for two waveform examples with the L7-4 transducer is compared with that of the GUI-AWE encoder (previous version). The examples include a Linear Frequency Modulated (LFM) waveform and a Discrete Prolate Spheroidal Sequence (DPSS) waveform. As shown in the table below, these examples demonstrate a **10-14 dB improvement in encoder performance**, over the VS1 GUI-based encoder.

waveform	NRMSE:VA	best gain	NRMSE: GUI VS	best gain	mprovement (dB)
DPSS(WB)	-27.8	0.6266	-13.4	0.00112974	-14.4
LFM chirp	-33.8	0.7562	-23.4	0.000615871	-10.4

DPSS Waveform Example The DPSS (Discrete Prolate Spheroidal Sequence) waveform example is illustrated in the figures below. Fig-1 shows the best tri-state approximation to the reference waveform as filtered by the transducer dynamics, and the error compared to the actual reference waveform.

Fig-2 shows results of the gain search, as a plot of normalized RMS error (NRMSE) vs. gain/scaling of the reference waveform. Here, the best error is seen at a reference signal gain/scaling of 0.6266. Note that in the figure, the “**Adjusted**” NRMSE allows an arbitrary delay and scaling to be applied during the error comparison to best align the comparison, as these could presumably be incorporated into the transmit sequence by means other than the encoder.

Fig-3 shows the results of the previous GUI-AWE encoder (VS1 algorithm). The improvement in NRMSE given by the new VA encoder in this example is dramatic (and visually obvious), by better than -14dB. Note that the actual gain/scalings for the previous GUI-AWE/VS1 encoder and the new CLAWE encoder are not directly compatible, as the previous encoder employs some automatic normalization that differs from the new Viterbi-based CLAWE.

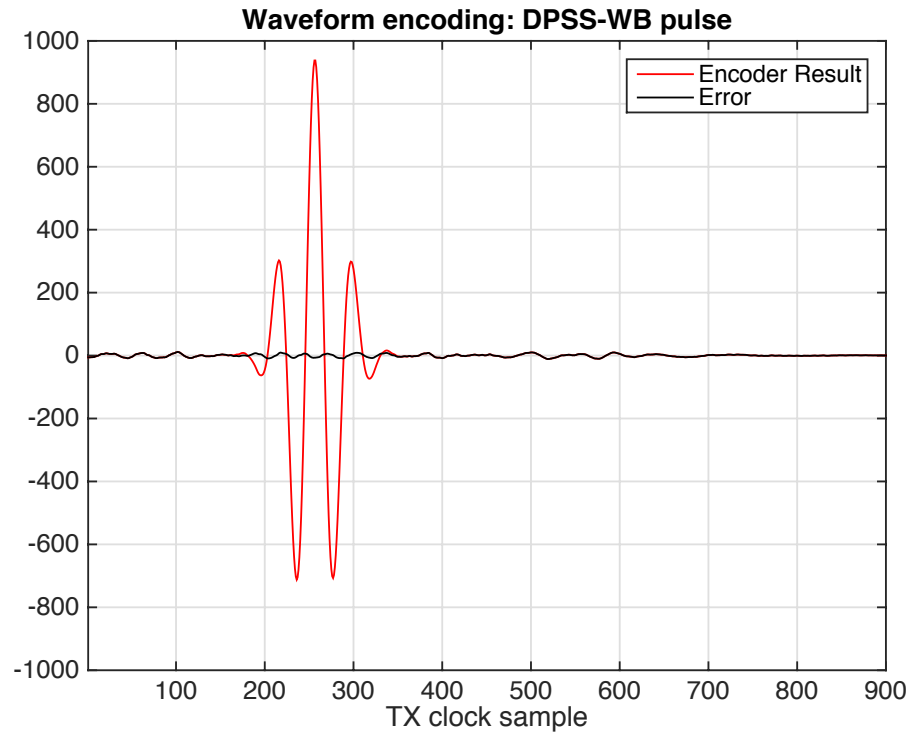


Fig-1 New encoder v2.1: DPSS pulse after reconstruction by transducer (L7-4).

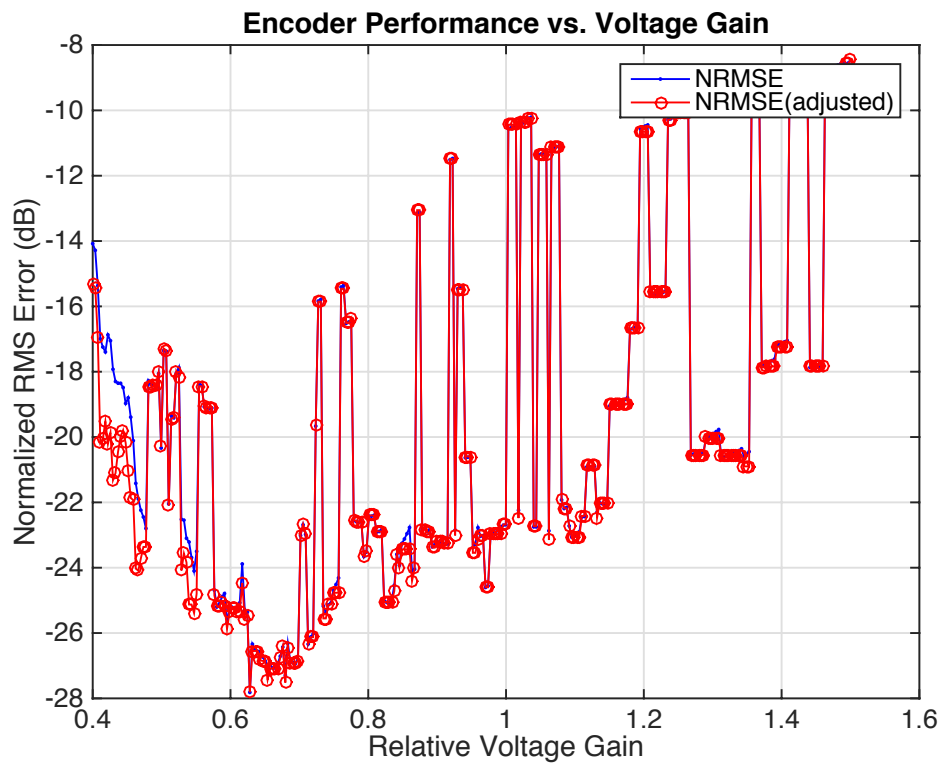


Fig-2 Gain search results for DPSS pulse encoding.

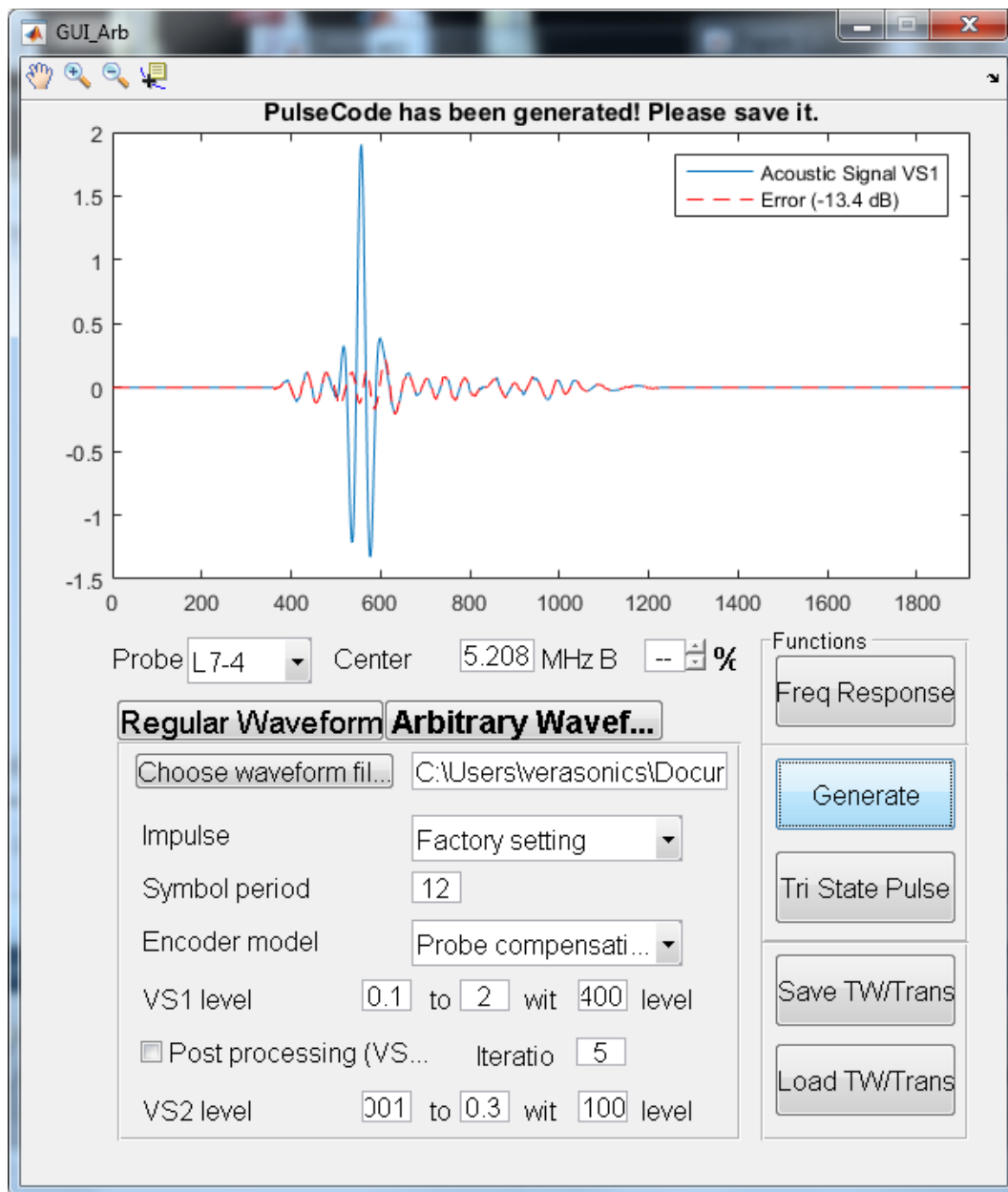


Fig-3 **Previous** (v1.0) GUI-based VS1 encoder reconstruction and error for DPSS pulse.

Appendix

Command Line Arbitrary Waveform Encoder ver. 2.1

Detailed Function Descriptions

File: **arbwavdev2.m**

Description:

Utility function for creating example analog reference waveforms, including Linear Frequency Modulation (LFM) Chirp, and Discrete Prolate Spheroidal Sequence (DPSS).

Inputs:

method = { 'reference.gausspulse', 'reference.lfm' }

See function source for parameter definitions associated with each method above.

Output: Data vector of waveform sampled at 250 MHz.

File: **encode_dp2.m**

Description:

Top-level example script to define user-specified “analog” transmit signals, setup encoder parameters, and invoke the encoder. Shows examples of defining the gain search vector. Contains encoding examples for two transmit signal types: Linear Frequency Modulation (LFM) Chirp, and Discrete Prolate Spheroidal Sequence (DPSS).

Inputs: Select example method by specifying the string variable: **testsig** [char] as noted below, and running the script `encode_dp2` in the workspace. Alternately, set the workspace variable **functionalityTestMode** as noted below for suggested parameter values.

Outputs: Results are saved in a .mat file:

Example for quick functionality verification (L11-5v transducer): set workspace variable to a character string value or numeric value as noted here:

```
>> functionalityTestMode = 'fast.2';
>> % or you may use the numeric value:
>> functionalityTestMode = 2;
>> encode_dp2; % This runs the test.
```

The result should show a plot labeled as “Computed signal” in panel 3 of a MatLab window labeled Figure 401.

Examples for transducer L11-5v: set workspace variable `functionalityTestMode` to a value of 1, 2, or 3 and run `encode_dp2` in the workspace.

Examples for transducer L22-14vX: set workspace variable `functionalityTestMode` to a value of 4 and run `encode_dp2` in the workspace.

Example encoder parameter settings are automatically specified depending on `functionalityTestMode` and may be inspected near the top of the script **encode_dp2.m**

Additional examples:

LFM Chirp Examples: `testsig = { 'lfm.L7-4.1', 'lfm.L7-4.2', 'lfm.L7-4.3' }`
Results are saved in .mat file: `encode_lfm_result.mat`

DPSS pulse examples: `testsig = { 'dpss.L7-4.1', 'dpss.L7-4.wb', 'dpss.L7-4.nb' }`
Results are saved in .mat file: `encode_dpss_result.mat`

File: getIRinfo.mDescription:

Retrieve impulse response for specified transducer from impulse response file.

Inputs: cirFileCode, testParam

cirFileCode: transducer name as character string

Supported transducers (new format):

'L11-5v'
'L22-14vX'

Legacy transducer (old format):

'L7-4'

testParam: this parameter is for testing and should be set to zero.

Outputs: IRstruct , clip

IRstruct: structure containing Impulse Response information.

clip: structure indicated extraction indices from raw estimate data.

File: nrmseVec_VS.pDescription:

Compute normalized RMS error between two vectors.

Inputs: vector1, vector2

Outputs: structure containing error metric.

File: viterbi_factor2.p

Description: Encoding, preprocessing and signal conditioning.

Inputs:

A. **method** = {[], 'initVoltage.01', 'approximate.resample'} (synonyms)

B. **paramStruct**: structure containing the following members:

symbolRateSetup: defines the Viterbi symbol pulse size in TX clocks. See the file **viterbi_symbol2.m** to see definitions and define more user settings. For examples, setting **symbolRateSetup**=7 defines a Viterbi symbol period of 3 transmitter clocks

refSignal: Structure defining reference/design analog signal to be encoded, with members:

x250: analog reference signal vector (double).

offsetFactor: reference signal zero pad amount at beginning of signal.

endpadFactor: reference signal zero pad amount at end of signal.

The zero-pad factors represent the fraction of the reference signal length, that defines the length of the zero pad amounts. For example, an **offsetFactor** of 0.5 will include leading zero pads that is 50% of the reference signal length.

amblesCode: Defines pre-amble and post-amble to Viterbi sequence, forcing zero-states to initial and/or final estimated transmitter tri-state sequence values.

regFact: Regularization factor to increase solution stability at the expense of introducing a controlled amount of error. Typical value is 1e-3 or smaller.

gVec: Gain search vector. This defines set of reference signal gain values to evaluate/optimize as a nuisance variable.

fMatchRatio: approximate implied gain of Viterbi model vector B. Typical value is 0.01, when empirical transducer impulse response units are receiver DAC counts, Viterbi model vector/filter has a passband gain of 0dB, and reference analog design signal has a value range of [-1.0 to 1.0].

solverMeth: Defines solver method for linear processing component. Use 'backslash' (faster execution), or 'pinv' (default, slower) if possible regularization issues are seen.

cirFileCode: Name of .mat file containing transducer impulse response definition. Impulse response must be stored in the .mat file as the structure **cirS**, where the impulse response vector (of class `double`) is defined in the member:

cirS.CIR.gateSignalModel (legacy format)

Alternately, impulse response data is loaded by specifying the character string variable `xdcr = <transducer name>`, currently valid for L11-5v and L22-14vX (new-format files in VerasonicsRoot\System), in parameter settings for specific reference signals; these can be inspected in the `testsig` switch in **encode_dp2.m**

Note: parameters above in the structure following the **method** parameter may also be specified as a comma-separated list, in the order defined above.

Outputs: structure variable **vitResult** with the following members:

vitResult.xOpt: Achieved approximation to the reference signal, at 1:cpp decimation rate.

vitResult.sEst: Estimated decimated transmitter tri-state sequence, at 1:cpp decimation rate.

vitResult.modelGain: defined as the product `fMatchRatio*gainBest`.

vitResult.gainBest: gain value in the vector of gains **gVec** that resulted in the lowest approximation error.

vitResult.fMatchRatio: The value of the input parameter `fMatchRatio` (defined above) used.

vitResult.bestNRMSEdB: normalized RMS error of the achieved approximation to the reference.

vitResult.bestNRMSEdBAdj: approximation error, similar to `bestNRMSEdB` but with allowance of least-squares adjustment of post-encoding scaling to best match the achieved/estimated and reference transmit signals. Thus this measure of accuracy assumes that the absolute level of the signal is not important.

vitResult.bestIndIndAdj: indexing to align approximated transmit signal and reference design signal for best match.

vitResult.txTriState250: (`int8`) Estimated tri-state sequence encoding vector defined as samples at the transmitter clock rate (250MHz). Use this sequence as input to the pulse code formatter tool, for example:

```
>> PC = twgen('trinary2gen3', double( vitResult.txTriState250 ));
```

vitResult.paramSet: parameter structure of Viterbi processing parameters, and intermediate gain search results.

File: **viterbi_symbol2.m**

Description:

User-definable (editable) script repository for parameters controlling symbol width and partitioning of the impulse response.

Inputs:

symbolRateSetup: numeric code labeling a set of parameter presets.

View the MatLab script file **viterbi_symbol2.m** to see various combinations of parameters related to Viterbi symbol period. Usage examples are given in the script **encode_dp2.m**. The following parameters are defined below in under “Outputs”.

Outputs:

cpp: Decimation/Interpolation rate applied to relevant signals, so that the Viterbi processing operates on a decimation rate. In this manner, each Viterbi symbol corresponds to a **cpp** number of transmit **clocks per pulse**.

Fv: Viterbi symbol rate in MHz, defined as the transmitter clock rate (in MHz) divided by the decimation rate: $F_v = 250/\text{cpp}$.

shortPulseFunc: The function name or function handle of the filter generation function that produces the Viterbi model vector.

FcFactored: Cutoff frequency used by filter generation function **shortPulseFunc**.

File: **cirest0701_code_acoustic-L74-23.mat**

Description:

Legacy-format transducer impulse response file example for Philips (r) L7-4 probe. The impulse response must be stored in the .mat file as the structure **cirS**, where the impulse response vector (of class `double`) is defined in the structure member:

Impulse response vector: **cirS.CIR.gateSignalModel**

Private/Utility Functions

Files: **isodd.p** , **alignvec03.p** , **makeAlphabet.p**, **nrmseVec_VS.p**

Description:

The above components are private/helper functions not currently intended for direct invocation by the user.

[End of Document]